



1. Conceptos y Herramientas Fundamentales

Terraform (Infraestructura como Código)

Declarativo e Idempotente: Permite definir y aprovisionar el centro de datos a través de código (HCL). En lugar de crear servidores manualmente (imperativo), tú 'declaras' cómo quieres que sea la arquitectura final. Terraform calcula la diferencia entre la realidad y tu código, y aplica los cambios necesarios garantizando que el resultado sea siempre exacto.

Docker (Contenedorización)

Aislamiento Ligero: A diferencia de las máquinas virtuales tradicionales que emulan hardware y un SO completo, Docker empaqueta el software y sus dependencias en 'contenedores'. Éstos comparten el Kernel del host mediante 'Namespaces' (aislamiento de procesos) y 'Cgroups' (limitación de CPU/RAM), siendo ultrarrápidos y consistentes en cualquier entorno.

Prometheus & PromQL (Observabilidad)

Métricas por Pulling: Prometheus es una Base de Datos de Series Temporales (TSDB) diseñada para fiabilidad. En lugar de que los servidores le envíen datos (Push), Prometheus va a buscarlos activamente (Pull). Utiliza PromQL, un lenguaje matemático avanzado que permite diseccionar telemetría masiva en tiempo real para generar alertas.

SaltStack (Gestión de Configuración)

Arquitectura Master/Minion: Herramienta de ejecución remota extrema basada en un bus de eventos cifrado (ZeroMQ). Permite aplicar configuraciones estandarizadas a miles de nodos simultáneamente. Garantiza una arquitectura 'Zero Trust' donde los minions deben ser criptográficamente aceptados en el Master antes de recibir cualquier orden.

2. Inteligencia PromQL y Alertas

Prometheus no usa bases de datos relacionales (SQL). Evalúa Time-Series (series temporales basadas en etiquetas) mediante álgebra sobre **Vectores Instantáneos** (el milisegundo actual) y **Vectores de Rango** (histórico).

A. REGEX & ENDPOINT HEALTH

```
up{job=~".*-node"} == 0
```

Lógica: La métrica up es generada internamente por Prometheus (1=Pull OK, 0=Timeout). Usamos la expresión regular =~".*-node" para vigilar dinámicamente cualquier contenedor o máquina que caiga, sin necesidad de harcodear IPs.

B. ÁLGEBRA VECTORIAL (RAM)

```
(1 - (node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes)) * 100 > 95
```

Lógica: Los agentes envían bytes crudos. PromQL evalúa algebraicamente dos series temporales distintas, realizando cálculos matemáticos al vuelo para disparar la alerta cuando el clúster cruza el 95% de uso real de RAM.

C. REGRESIÓN PREDICTIVA

```
predict_linear(node_filesystem_free_bytes{mountpoint="/"}[6h], 24*3600) < 0
```

Lógica: Convierte la consulta en un Vector de Rango [6h]. Analiza la velocidad exacta a la que se llena el disco duro en las últimas 6 horas y aplica matemáticas predictivas para alertar si nos quedaremos sin espacio en las próximas 24 horas.

D. VENTANAS DE CONTADORES

```
increase(node_vmstat_oom_kill[5m]) > 0
```

Lógica: Los contadores (Counters) siempre crecen. Si el Kernel aniquila un proceso por falta de RAM, el contador sube. `increase()` evalúa solo los eventos nuevos detectados en la ventana móvil de los últimos 5 minutos.

3. Operaciones Core (CLI)

COMANDO SHELL	ARQUITECTURA / BAJO EL CAPÓ	CONTEXTO
terraform apply	Traduce el código declarativo (.tf) en llamadas a las APIs (AWS, Docker) para crear los recursos de infraestructura.	ALTO
docker exec -it <vm>	Accede directamente al socket interactivo (bash) del contenedor sin requerir instalar un demonio SSH.	BAJO
salt-key -A -y	Valida las claves PKI pendientes en el Master, cerrando el anillo de confianza Zero Trust con los Minions.	MEDIO
salt '*' test.ping	Verifica la latencia del bus ZeroMQ exigiendo respuesta en tiempo real a todos los agentes conectados.	NULO
kill -HUP <pid>	Señal de SO que fuerza a Prometheus a recargar su archivo YAML en memoria sin tirar la base de datos.	MEDIO

Pipeline DevOps

1. Red Híbrida L2
ip link add name Lab type bridge
2. IaC (Terraform)
terraform init && terraform apply
3. Control (SaltStack)
salt-key -A && salt '*' state.apply
4. TSDB (Prometheus)
systemctl start prometheus
5. Visualización (Grafana)
curl -X POST .../api/dashboards

IaC: terraform.tf

HCL

```
# Definición declarativa del Contenedor Master
resource "docker_container" "vm_master" {
  name      = "vm-master"
  image     = "ubuntu:24.04"

  # IP Estática Inamovible (.40)
  networks_advanced {
    name      = docker_network.internal_net.name
    ipv4_address = "172.28.0.40"
  }

  # Volúmenes (Persistencia de BD Temporal)
  volumes {
    volume_name      = docker_volume.prometheus_data.name
    container_path = "/var/lib/prometheus"
  }

  # Inyección de configuración (Upload al vuelo)
  upload {
    source = "${path.cwd}/../monitoring/prometheus.yml"
    file   = "/etc/prometheus/prometheus.yml"
  }
}
```

EXPORTERS (ENDPOINTS)

Node Exporter (TCP 9100) Micro-agente pasivo de Prometheus que lee del Kernel Linux (/proc, /sys) para exponer métricas de SO (CPU, RAM, red).
cAdvisor (TCP 8080) Monitor de Google montado en docker.sock. Extrae telemetría individual contenedor por contenedor.
Blackbox Exporter Auditor sintético. Simula a un usuario final lanzando Pings y peticiones HTTP/S para validar caídas reales.
Grafana Server (TCP 3000) Capa de visualización pura. Ejecuta las consultas PromQL complejas y las convierte en gráficas operativas.